ChatGPT

JavaScript Interview Questions by Topic

1. Core Language Concepts (Variables, Data Types, Coercion, Operators)

Basic

- **What are the different data types in JavaScript?** – List the primitive types (e.g. string, number, boolean, etc.) and the `Object` type [1]. Explain how `null` and `undefined` differ [2].
- **What is the difference between `==` and `===` in JavaScript?** – Describe how the loose equality operator performs type coercion and how strict equality avoids it [3]. When would you use one over the other?
- **How do `var`, `let`, and `const` differ when declaring variables?** – Compare their scope rules and reassignability (function vs block scope, hoisting behavior, and immutability of bindings) [4].
- **What values are considered "falsy" in JavaScript?** – Define *falsy* values (e.g. `false`, `0`, `""`, `null`, `undefined`, `NaN`) which behave as false in boolean contexts [5]. Give an example of a truthy vs falsy value in an `if` statement.
- **What is type coercion?** – Explain how JavaScript converts values between types implicitly (e.g. number-to-string conversion) and give an example of a surprising result of coercion (such as `"5" - 2 or "5" + 2`). Why is understanding coercion important?

Intermediate

- **Are function arguments passed by value or by reference in JavaScript?** – Discuss how primitives are passed by value and objects are passed by value of the reference (i.e. the reference is copied). Illustrate with an example to show that reassigning a function argument inside does not affect the original variable.
- **How does the `typeof` operator work?** – What does `typeof` return for various types (string, number, object, function, etc.) and what are some quirks (for example, `typeof null`)? How can `typeof` be useful for type checks?
- **What is `NaN` in JavaScript?** – Explain what `NaN` (Not-a-Number) represents, what its type is, and how to reliably check if a value is `NaN` (e.g. using `Number.isNaN`). Why does `NaN === NaN` evaluate to false?
- **Explain** dynamic typing **in JavaScript.** – Discuss what it means that JavaScript is dynamically (and loosely) typed. How can this lead to runtime type errors? Include an example of how a variable's type can change and why static typing (TypeScript) can help catch errors.
- **What are Symbols in JavaScript?** – Describe the ES6 `Symbol` type and its purpose (unique identifiers that don't clash with other property keys). In what scenarios might you use a `Symbol`?

Advanced

- **What is a BigInt and when would you use it?** – Explain the `BigInt` type (ES2020) for representing integers beyond the `Number` precision limit. How do you create a BigInt, and what are some limitations of using BigInts versus Numbers?

- **Explain the concept of the *Temporal Dead Zone*.** – In the context of `let` and `const`, define the Temporal Dead Zone (TDZ) and describe what happens if you access a `let`/`const` variable before its declaration. Why does this design help prevent certain bugs [6]?
- **How does JavaScript's garbage collection work (at a high level)?** – Outline the basics of memory management in JS: the mark-and-sweep algorithm for garbage collection, how unreachable objects are collected, and why developers usually don't manually manage memory.
- **Is everything in JavaScript an object?** – Discuss the relationship between primitives and objects. While primitives like strings and numbers are not objects, explain how they can behave like objects (e.g. accessing `String` methods) through boxing. Mention that functions are actually objects (callable objects).
- **What is *automatic semicolon insertion* and can it cause issues?** – Describe JavaScript's rules for inserting semicolons in certain cases automatically. Give an example (such as a return statement on one line and a value on the next) where omitting a semicolon can lead to unintended behavior.

2. Functions, Scope, and Closures

Basic

- **How do function declarations and function expressions differ?** – Compare writing a function as `function myFunc() {}` versus `const myFunc = function() {}` [7]. Discuss hoisting (declared functions are hoisted, whereas function expressions act like any variable) and when each form might be used.
- **What is *hoisting* in JavaScript?** – Define hoisting as JavaScript's behavior of moving declarations to the top of their scope during compilation [8]. Explain how `var` declarations and function declarations are hoisted, but `let`/`const` and function *expressions* are not initialized before execution.
- **What are the different scopes for variables in JavaScript?** – Explain function scope vs block scope vs global scope [9]. For example, describe where a variable declared with `var` inside a function is accessible, and how `let`/`const` introduce block-level scope.
- **What is an IIFE (Immediately Invoked Function Expression)?** – Describe the pattern of defining and immediately executing a function: e.g. `(function(){ /*...*/ })()`. Why was this pattern commonly used (to create a private scope and avoid polluting the global namespace)?
- **What is a higher-order function?** – Define a higher-order function as one that either takes functions as arguments or returns a function as a result. Provide a simple example (e.g. a function that takes a callback, or a function that returns a new function for addition) to illustrate the concept [10].

Intermediate

- **What is a closure, and how/why would you use one?** – Explain that a closure is a function that remembers its outer variables and scope even when that function is executed outside its original scope [11]. Provide an example (such as a function factory or a simple counter function) to demonstrate how closures allow data privacy or stateful function behavior.
- **How does the `this` keyword work inside a function?** – Describe what `this` refers to in different calling contexts (method call, plain function call, constructor call, `call/apply` invocation) [12]. For example, in a method `obj.method()`, `this` is `obj`, whereas in a standalone function call `this` is global (or `undefined` in strict mode). Mention how context is determined at call time.

- **What are `call`, `apply`, and `bind` used for?** – Explain that `func.call(thisArg, ...args)` invokes a function with an explicit `this` value and arguments, `func.apply(thisArg, argsArray)` does the same with an array of args [13], and `func.bind(thisArg)` returns a new function permanently bound to a `this` value. When might you use `bind` (e.g. to set `this` for a callback)?
- **What's the difference between a function statement and a function assigned to a variable?** – Discuss the distinction between `function foo() {}` and `var foo = function() {}` in terms of hoisting and naming [14]. For instance, a function declaration is hoisted with its name, whereas a function expression is not usable before it's defined.
- **How can a function accept an indefinite number of arguments?** – Describe the ways to handle variable arguments: the older `arguments` object available within any function, and the modern ES6 rest parameter syntax (`function f(...args)`). When would you prefer using rest parameters over the `arguments` object?

Advanced

- **What is *currying* in JavaScript?** – Explain currying as transforming a function that takes multiple arguments into a sequence of functions that each take a single argument [15]. For example, a curried function `add(a)(b)` that adds two numbers in two calls. Discuss benefits of currying (such as partial application or configuration of functions).
- **How do generators work in JavaScript?** – Describe generator functions (declared with `function*`) and the `yield` keyword. How do generators differ from normal functions? Illustrate how calling a generator returns an iterator that can produce values lazily, and mention a use case for generators (e.g. custom iteration or cooperative multitasking).
- **What is the *scope chain*?** – In the context of nested functions, explain how identifier lookup works: first local scope, then outer scope(s), up to global scope. Use a closure example to show how a function can access variables from a parent scope even after that parent function has returned.
- **Can you give an example of *memoization* in JavaScript?** – Describe what memoization is (caching function results to avoid repeated computation). Outline how you could implement a simple memoization for a pure function (storing results in a closure or object). Discuss when this technique is useful for performance.
- **How did ES6 arrow functions change the behavior of `this`?** – Explain that arrow functions do **not** have their own `this` binding; instead, they inherit `this` from the enclosing lexical scope [16]. This is useful for callbacks (e.g. in array methods or timers) where you want to maintain the outer `this`. Also note that because of this, you cannot change an arrow function's `this` with `call` or `bind`.

3. Objects, Prototypes, and `this`

Basic

- **How do you create an object in JavaScript?** – Describe literal syntax (e.g. `const obj = { name: "Alice", age: 25 }`) [17] and other ways like using the `Object` constructor or classes. Give an example of defining an object with properties and their values.
- **What is the `this` keyword in JavaScript?** – Explain that `this` refers to the "owner" of the function or the object that is executing the current code [18]. Provide a basic example where `this` inside a method refers to the object instance. Mention that the value of `this` depends on how a function is called (it's not static).
- **How are primitive values and objects different in JavaScript?** – Highlight that primitives (string, number, boolean, etc.) are immutable and compared by value, while objects (and arrays/

functions) are reference types compared by reference. For example, explain why two different `{}` objects are not equal even if they have the same properties, whereas two identical string literals are equal.
- **What's the difference between dot notation and bracket notation for object properties?** – Provide examples of accessing properties with `obj.prop` versus `obj["prop"]`. Explain when bracket notation is required (e.g. property name is dynamic, or not a valid identifier). Also mention that bracket notation allows property names with spaces or special characters.
- **How can you check if an object has a certain property?** – Discuss using the `in` operator (e.g. `"key" in obj`) and the `obj.hasOwnProperty("key")` method. Briefly note the difference between own properties and inherited properties when using these checks.

Intermediate

- **Explain how prototypal inheritance works in JavaScript.** – Describe that every object has an internal link to a prototype (`__proto__`) which is another object [19]. When a property is not found on an object, the engine looks up the prototype chain. Use an example: if `dog` inherits from `animal`, a call to `dog.eat()` will find `eat` on `animal.prototype` if not on `dog` itself. Compare this to class inheritance (mention that ES6 classes are essentially syntax sugar over prototypal inheritance).
- **What is the prototype property of a constructor function?** – Explain that for a function `FunctionName`, `FunctionName.prototype` is an object that will become the prototype of any objects created by that constructor (via `new`). By adding methods to a constructor's prototype, all instances share those methods. For instance, if `Circle.prototype.area = ...`, then any `new Circle()` can call `.area()`.
- **How does `this` behave in different contexts?** – Discuss `this` in methods vs. standalone functions vs. constructors [12]. For example: inside an object's method, `this` is the object; if that method is detached and called as a regular function, `this` becomes global/undefined. In an event listener, `this` might refer to the DOM element. Mention the common pitfall where a callback loses the original `this` and techniques like `bind` or arrow functions to preserve it.
- **What is the difference between calling a function with `new` vs without?** – Using an example like `function Person(name) { this.name = name; }`: explain that `Person("Alice")` as a regular call will do something different (or throw an error if strict mode, since `this` would be undefined) compared to `new Person("Alice")` which creates a new object, sets its prototype, and binds `this` to that new object [20]. In short, `new` is used to instantiate objects via constructor functions or classes.
- **How would you clone or copy an object in JavaScript?** – Discuss the concepts of shallow copy vs deep copy. For shallow copies, mention using `Object.assign({}, obj)` or spread `{ ...obj }` for plain objects. For deep copying, one might use JSON serialization (`JSON.stringify` / `parse`) or structured clone or utility libraries. Ask about the implications: shallow copy only copies references for nested objects, which can lead to unintended mutations.

Advanced

- **What are the differences between prototypal inheritance and classical inheritance?** – Compare JavaScript's prototype-based model to class-based inheritance in languages like Java or C++ [21]. For example, explain that in prototypal inheritance objects inherit directly from other objects, allowing for more dynamic and flexible inheritance, whereas classical inheritance involves hierarchies of classes. Discuss how ES6 classes are implemented on top of prototypes and that JavaScript remains prototypal under the hood.
- **Why is extending built-in JavaScript objects generally discouraged?** – Explain what it means to *monkey-patch* or add properties to built-in prototypes (like `Array.prototype` or

`Object.prototype`) and why this can be problematic [22]. (It can lead to conflicts, break iteration, or impact other code unintentionally.) Mention that one should prefer composition or utility functions over modifying built-ins.
- **What is a *mix-in* and how can you achieve it in JavaScript?** – Describe the pattern of borrowing methods from one object or class to another (as an alternative to inheritance). For instance, copying properties from a source object into a target object (using `Object.assign` or via prototype manipulation) to add functionality. Discuss use cases for mix-ins in JS (such as applying common functionality to different object prototypes).
- **How do ES6 classes handle inheritance under the hood?** – Although we use the `class extends` syntax, explain that it sets up the prototype chain internally (the subclass's prototype's `__proto__` points to the superclass's prototype). Mention the role of `super` in constructor calls and method overriding. This question checks understanding that classes are not a new inheritance model but syntactic sugar over prototypes.
- **What are *private* class fields in modern JavaScript?** – Introduce the concept of class fields starting with `#` (ES2022) which are truly private (not accessible outside the class definition). How do these differ from the old convention of pseudo-private properties (like prefixing with `_`)? What benefits do private fields provide for encapsulation?

4. Asynchronous JavaScript (Callbacks, Promises, Async/Await)

Basic

- **What is a callback function in JavaScript?** – Define a callback as a function passed into another function to be executed later [23]. For example, in `setTimeout(fn, 1000)`, `fn` is a callback executed after the delay. Explain how callbacks allow asynchronous behavior (the callback runs after some work completes) and mention a common use like handling responses in network requests.
- **What problems can arise from heavy use of callbacks, and how were they addressed?** – Discuss **callback hell** (the "pyramid of doom" with deeply nested callbacks that become hard to read and maintain) and inversion of control issues. Explain that Promises were introduced to improve code readability and error handling for async operations [24]. You might point out how `.then/.catch` chains or `async/await` (built on promises) make asynchronous code look more sequential.
- **What is a Promise in JavaScript?** – Describe a Promise as an object representing the eventual completion or failure of an asynchronous operation and its resulting value. Explain the basic states of a Promise (pending, fulfilled, rejected) and how you attach handlers with `.then()` for success and `.catch()` for errors [25]. Why do promises make asynchronous code easier to manage than callbacks?
- **How do you convert a callback-based function to use Promises?** – Conceptually, describe wrapping an async operation that uses a callback (like Node's `fs.readFile`) inside a new Promise, calling `resolve` on success and `reject` on error. This tests understanding of bridging old-style callbacks to promise style.
- **What is the difference between synchronous and asynchronous functions?** – Explain that synchronous code blocks the execution until it finishes (each statement completes before the next one starts), whereas asynchronous code can start something now and finish it later, allowing the program to continue in the meantime [26]. Provide a simple analogy or example (e.g., reading a file synchronously vs asynchronously) to illustrate the difference.

Intermediate

- **How do you handle errors in Promise chains or async/await?** – Explain the error handling model: with promises, you handle errors in a `.catch()` at the end of a chain (or by returning rejected promises) – unhandled rejections could be a problem. With `async/await`, you use try/catch around the await calls just like synchronous code. Possibly mention that an uncaught exception in an async function automatically causes the returned Promise to reject.
- **What is `async/await` and how does it relate to Promises?** – Describe `async` functions (ES2017) as a syntactic sugar over promises. Inside an `async` function you can `await` a promise to pause execution until it resolves, making asynchronous code look linear. Provide a brief example of an async function using `await` instead of explicit `.then` calls. Emphasize that `await` can only be used inside `async` functions (or modules, with top-level await).
- **Give an example of making an HTTP request in JavaScript and handling the result asynchronously.** – You might talk about using the Fetch API which returns a promise, e.g. `fetch(url).then(res => res.json()).then(data => ...)`. Or using `async/await`: `const res = await fetch(url)`. This question gauges familiarity with real-world async tasks (like network calls) and using promises to handle them.
- **What are some ways to avoid "callback hell"?** – Besides promises/async-await, mention techniques like **function modularization** (splitting nested callbacks into separate functions), using **control flow libraries** (in older times) or simply keeping code flatter by returning promises. Essentially, ensure the candidate recognizes the problem and knows high-level strategies to deal with deeply nested async callbacks.
- **How can you execute multiple asynchronous operations in parallel and get notified when they all complete?** – Expect an answer involving **Promise.all()**. For example, "If I have three independent promises, I can use `Promise.all([p1, p2, p3])` to get a new promise that fulfills when all three are fulfilled (or rejects if any fail)." Possibly also mention `Promise.race` or other combinators as a plus.

Advanced

- **What is "callback hell" and how can it be avoided?** – Have the candidate describe the scenario of multiple nested callbacks making code hard to maintain [27]. Solutions should include: using promises to flatten the structure, using async/await to write code that looks synchronous, or using event emitters/observables depending on context. This is similar to above but expecting a deeper discussion or recognition of an example callback pyramid.
- **Explain the event loop's role in asynchronous JavaScript.** – Here the candidate should connect to how callbacks and promises are managed under the hood. For instance: the JavaScript runtime enqueues callback functions in the task queue to be executed by the event loop once the call stack is free [28]. Specifically, mention how **promises' `.then` callbacks go to the microtask queue**, which gets priority over the macrotask queue (setTimeout, etc.). This question checks if they understand why, for example, a `setTimeout(fn, 0)` might run after a promise's `.then` callback, even if both are scheduled at the same time.
- **What are generator functions, and how can they be used for asynchronous programming?** – Discuss how generator functions (`function*`) can yield execution, and how libraries (or earlier approaches) used generators + promises (with a runner like `co`) to achieve async/await-like syntax before `async/await` existed. This is advanced knowledge: expecting mention of iterating over a generator and yielding promises (as an alternative to async/await).
- **What is the difference between microtasks and macrotasks in the JavaScript concurrency model?** – Explain that certain asynchronous callbacks are scheduled as *microtasks* (e.g. promise callbacks, `queueMicrotask()`) which run immediately after the current function completes, before the event loop continues with other tasks. In contrast, *macrotasks* (regular callbacks from

`setTimeout`, `setInterval`, DOM events, etc.) are handled in the task queue and may be delayed until after rendering. This affects the order in which asynchronous operations execute.
- **How would you implement a *retry mechanism* for a promise-returning function?** – This tests the ability to manage async control flow. Expect an outline like: "Call the function, and in the `.catch` of the promise, decide to call it again after a delay or a certain number of retries." Or using recursion with async/await: an `async function retry(fn, retries)` that calls `await fn()` and catches to retry. This demonstrates understanding of promise control beyond basics.

5. DOM Manipulation and Events

Basic

- **What is the DOM?** – Define the Document Object Model as a programming interface for HTML/XML documents. It represents the page structure as a tree of nodes/elements that JavaScript can interact with. Essentially, explain that the DOM allows JS to access and manipulate the content, structure, and styles of a web page.
- **How do you select an element from the DOM using JavaScript?** – Provide examples of common DOM selection methods: `document.getElementById("id")`, `document.getElementsByClassName("class")`, `document.getElementsByTagName("tag")`, and modern selectors like `document.querySelector` / `querySelectorAll`. For instance: *"To select an element with id* `container`*, you can do* `const el = document.getElementById('container')`*."*
- **How can you change the content or style of an HTML element with JavaScript?** – For content, mention `element.textContent` or `element.innerHTML` (and note the differences/risks of innerHTML). For style, mention setting properties on `element.style` (e.g. `element.style.backgroundColor = 'red'`). Possibly touch on using CSS classes with `classList.add/remove` as a preferred method to alter styles.
- **How do you attach an event handler to a DOM element?** – Explain the use of `addEventListener`. For example: *"You can do* `button.addEventListener('click', eventHandler)` *to run a function when a button is clicked."* Contrast this with older methods like `onclick` properties, and mention that `addEventListener` allows multiple handlers and more control.
- **What is the difference between an HTML attribute and a DOM property?** – Clarify that attributes are defined in HTML (the initial markup) whereas properties are values on the DOM node objects. For example, the `value` of an `<input>` element: the attribute defines the initial value, and the DOM property `inputElement.value` reflects the current value which can change. Many attributes correspond to properties (id, class, etc.), but not all DOM properties have corresponding attributes [29].

Intermediate

- **What is *event bubbling* (and capturing) in the DOM?** – Describe the event propagation model: in capturing, an event travels from the window down to the target element; in bubbling, it goes from the target element back up to the window [30]. Explain that by default most events bubble up. For example, a click on a button will first run handlers on the button, then on its parent, and so on, unless `event.stopPropagation()` is used.
- **How is the `DOMContentLoaded` event different from the `load` event?** – Explain that `DOMContentLoaded` fires when the initial HTML document has been completely loaded and parsed (**without** waiting for stylesheets, images, and subframes to finish loading), whereas the

`load` event on window fires later, when *all* resources (images, scripts, etc.) have finished loading [31]. This influences when you can safely execute DOM manipulation scripts.

- **What is the *event object* passed to event handlers?** – Describe that when an event occurs, an event object (e.g. `MouseEvent` for clicks) is passed to the handler. It contains information about the event (like `event.type`, `event.target`) and methods (like `preventDefault()` and `stopPropagation()`). For instance, `event.target` is the element that actually triggered the event (the deepest target), which is often useful inside a generic handler.
- **How would you implement *event delegation*?** – Define event delegation as attaching a single event listener to a parent element and using it to handle events on its child elements (leveraging event bubbling) [32]. For example, instead of adding a click listener to every list item, attach one listener to the parent `<ul>` and determine which `<li>` was clicked by checking `event.target`. This improves performance and is useful for dynamically added elements.
- **What is the difference between `event.preventDefault()` and `event.stopPropagation()`?** – Explain that `preventDefault()` stops the default action of an event (e.g. preventing a form submission or link navigation), whereas `stopPropagation()` stops the event from bubbling up to parent elements. Mention that these are often used together when handling events like form submissions via AJAX (prevent the submit and stop it from bubbling).

Advanced

- **Explain how you would optimize DOM updates for better performance.** – Discuss techniques such as **batching DOM changes** (making multiple changes together to avoid repeated layout/reflow), using *document fragments* for multiple insertions, avoiding unnecessary DOM lookups by caching references, and minimizing layout thrashing (e.g., reading DOM values and writing DOM values in separate batches). Possibly mention using `requestAnimationFrame` for visual updates or debouncing rapid DOM updates.
- **What is the Shadow DOM?** – Define the Shadow DOM as a web components feature that allows encapsulation of DOM and styling. It's a "DOM within DOM" for an element, that isn't accessible from the main document DOM, preventing style leakage. Ask why it's useful: for creating custom elements/web components with isolated markup and style (so that styles or scripts don't inadvertently affect the internal implementation of a component).
- **How do you create and dispatch custom events?** – Describe using the `CustomEvent` constructor to create an event with a name and optional detail data (e.g. `const event = new CustomEvent('myEvent', { detail: {x: 10} });`) and then dispatching it with `element.dispatchEvent(event)`. Explain a scenario where you might use custom events (communication between components or modules that aren't directly connected).
- **Why might directly manipulating the DOM in a large application be challenging, and how do frameworks address this?** – This is more open-ended: expected answer might mention that manual DOM manipulation can get complex as the app grows (keeping state in sync with UI is hard). Frameworks like React/Vue use a **virtual DOM** or reactive binding to efficiently update the UI when the state changes, rather than manual DOM calls. Essentially, test if the candidate knows the broader reason frameworks abstract direct DOM manipulation (to improve reliability and performance).
- **What are Mutation Observers?** – Explain that MutationObserver is a Web API that lets you watch for changes in the DOM tree (e.g., elements added/removed or attributes changed) asynchronously. Mention a use case, such as implementing custom logic when the DOM changes or polyfills that need to react to added elements. This checks knowledge of less common but powerful DOM features.

6. The Event Loop and Concurrency Model

Basic

- **What is the JavaScript event loop?** – Provide a high-level description: the event loop is a mechanism that allows JavaScript (which runs on a single thread) to perform non-blocking operations by offloading tasks (like I/O) and queueing callbacks [28] . Explain that the event loop continuously checks the call stack and task queues (macro and micro) and processes events/callbacks when the call stack is free.
- **Why is JavaScript single-threaded, and what does that mean for concurrency?** – Explain that JavaScript (in browsers and Node's main thread) runs one operation at a time (one call stack). Concurrency is achieved not by parallel threads executing JS, but by the event loop coordinating asynchronous events. This means two pieces of JS code can't run at exactly the same time in one thread – which simplifies things (no data races) – but long-running tasks will block the thread.
- **What are the *call stack* and *task queue* (message queue) in the event loop?** – Define the call stack as the data structure that tracks what function is currently running and its callers. The task queue (or message queue) is where asynchronous callbacks are placed to be executed. For example, when a `setTimeout` timer completes, its callback goes to the task queue, and the event loop will eventually move it to the call stack when it's its turn [28] . This question expects an explanation of how an async callback eventually gets executed after the current code finishes.
- **Can JavaScript perform two operations at the exact same time?** – The expected answer is "not in the same thread." JavaScript achieves a form of concurrency via the event loop by quickly switching between tasks, but it's not parallel in a single thread. If needed, mention that browsers and Node can use background threads (for I/O or web workers) but the JS code you write in one main context is single-threaded.
- **Why might a `setTimeout(fn, 0)` not execute immediately?** – Explain that even a zero-delay timeout is not instantaneous – the callback is put into the event queue and will only run after the current call stack is clear (and after any previously queued events). Essentially, `setTimeout(fn,0)` schedules `fn` to run *soon*, but not until the JS engine has finished the current execution and processed pending microtasks. This reveals understanding of the queueing mechanism.

Intermediate

- **What is the difference between *macrotasks* and *microtasks*?** – Define macrotasks as tasks scheduled by mechanisms like `setTimeout`, `setInterval`, DOM events, or I/O callbacks. Microtasks are typically scheduled by promises (Promise callbacks `.then/.catch`) or `queueMicrotask`. The key difference: microtasks are processed immediately after the current script execution ends, **before** any macrotasks (thus, before rendering and before next event loop tick). Mention that this is why promise callbacks often fire before setTimeout callbacks scheduled at the same time.
- **How does the event loop in Node.js differ from the browser event loop?** – Note that Node.js has additional phases in its event loop (timers, pending callbacks, I/O poll, check (for `setImmediate`), close callbacks, etc.). While the core concept is similar, Node's loop is tuned for server tasks. Also mention that Node's event loop handles things like filesystem and network events differently and that Node allows offloading work to a thread pool (for file I/O or CPU-heavy tasks via `worker_threads` or C++ addons).
- **Explain a scenario where the event loop might *block*.** – For example, a while loop or a huge computation in JavaScript will block the event loop (no other callbacks can run until it finishes). Ask how to mitigate this (e.g. break the task into smaller chunks, use `setTimeout` to yield, or

use Web Workers for true parallelism on the web). This tests understanding of the limitations of single-threaded execution.

- **What is *process.nextTick* in Node.js?** – Discuss that `process.nextTick` in Node inserts a callback to be run **before** the next event loop tick – effectively, it's like a microtask (even higher priority than promise microtasks in Node's event loop, as it runs next tick before I/O). It's often used to schedule code to run after the current function but before any I/O events. (This is fairly advanced and Node-specific, showcasing deep event loop knowledge.)
- **How does asynchronous code improve performance if JavaScript is single-threaded?** – The answer should convey that while JavaScript itself runs one thing at a time, async code allows waiting for external operations (like network requests or file reads) without blocking the thread. The program can do other work or respond to other events while the async operation is in progress. Essentially, non-blocking I/O and async APIs let JavaScript handle many tasks (e.g., thousands of network sockets) seemingly in parallel, which is critical for performance on the web [33].

Advanced

- **Can you outline the phases of Node.js's event loop?** – Expected answer: Node's event loop has multiple phases – timers (execute `setTimeout`/`setInterval` callbacks), pending callbacks (I/O callbacks deferred to next loop), idle/prepare (internal), poll (retrieve new I/O events, execute I/O callbacks), check (execute `setImmediate` callbacks), close callbacks. This is quite advanced; an interviewee might list a few key phases and mention how `setImmediate` vs `setTimeout` differ in when they execute.
- **What is a *race condition* and can it occur in front-end JavaScript?** – While traditional race conditions are less of an issue in single-threaded JS, discuss scenarios where ordering issues occur: e.g. two asynchronous operations finishing in an uncertain order. Or race conditions in web workers or concurrent environments. For front-end: maybe loading data and then rendering assuming a certain order – if promises resolve out of order, that's a race condition. This tests conceptual understanding of concurrency issues.
- **How do Web Workers affect the JavaScript concurrency model?** – Explain that Web Workers allow creating additional JS threads that run in parallel to the main thread. They communicate via message passing (posting messages back and forth). This lets you offload heavy computations to a worker thread so as not to block the main UI thread. In system design terms, it introduces true multi-threading to JS at the cost of no shared mutable memory (except via SharedArrayBuffer). This is advanced knowledge about leveraging concurrency beyond the event loop.
- **Why might the order of output differ when using Promises vs `setTimeout`?** – This question expects an answer about microtask vs macrotask queues. For example: *"If you do* `Promise.resolve().then(()=> console.log('Promise'))` *and* `setTimeout(()=> console.log('Timeout'), 0)`*, the promise callback logs first because promise callbacks are microtasks which run before the event loop moves to the next macrotask like* `setTimeout`*."* It's a specific scenario to test understanding of task ordering.
- **If you have long-running JavaScript code (say, crunching numbers), how can you prevent it from freezing the page?** – Look for answers like: *"break the work into smaller chunks and schedule them with* `setTimeout` *or* `requestAnimationFrame` *so that the event loop can process user events in between,"* or *"use a Web Worker to move the computation off the main thread."* This indicates an understanding of cooperating with the event loop to keep the UI responsive.

7. ES6+ and Modern JavaScript Features

Basic

- **What are arrow functions (`=>`) and how do they differ from traditional functions?** – Explain that arrow functions are a shorter syntax introduced in ES6 for writing functions [34]. They lexically bind `this` (so `this` is inherited from the parent scope, rather than depending on how the function is called). Also mention they cannot be used as constructors and do not have their own `arguments` object. For example, compare `function(x){ return x * 2; }` with `(x) => x * 2`.
- **What are template literals in ES6?** – Describe template string syntax using backticks `` ` `` that allows embedding expressions with `${...}` [35]. Show a quick example: `` `Hello, ${name}!` `` produces a string with the value of `name`. Mention that template literals can span multiple lines and that they support features like tagged templates for advanced usage.
- **Can you give an example of array or object destructuring?** – Explain the destructuring syntax introduced in ES6 [36]. For arrays: e.g. `const [first, second] = [10, 20];` assigns `10` to `first`, `20` to `second`. For objects: e.g. `const {name, age} = person;` pulls out properties into variables. Discuss how this can make extracting values more convenient than manual indexing or property access.
- **What do the `...` spread and rest operators do?** – Describe that the `...` syntax has two main uses [37]. Spread: expanding an iterable or object in places where zero or more elements/keys are expected (e.g. `myFunc(...array)` or `{...obj}` to shallow-copy properties). Rest: collecting multiple elements into an array or parameters into a single parameter (e.g. `function f(...args)`). Provide a simple example of each and why they're useful (e.g. merging arrays, cloning objects, variadic functions).
- **What new data structures were introduced in ES6?** – Mention **Map** and **Set** (and their weak counterparts). A Map is like an object but allows keys of any type and has methods like `set`, `get`. A Set is a collection of unique values. These were added in ES6 to supplement plain objects and arrays for certain use cases (e.g. needing key order, or efficiently checking existence of an item). An example: using a Set to store a unique list of items, or a Map to reliably map object keys to values without key conversion to strings.

Intermediate

- **How do ES6 classes improve upon prototypal inheritance?** – Note that ES6 `class` syntax provides a cleaner, more familiar way to create constructor functions and manage inheritance using `extends` [4]. Under the hood it's still prototypes, but classes bring features like constructors, super calls, and static members. Ask about differences: for example, classes are not hoisted the way function declarations are, and methods defined in a class are non-enumerable. It's essentially syntactic sugar, but helps organize code.
- **What is the difference between *default exports* and *named exports* in ES6 modules?** – In ES6 modules, you can export multiple named values (e.g. `export function A()...`) and import them with curly braces, or have one default export (`export default function(){}` or value) which can be imported without braces, using any name. Ask for an example of each and when you might use one or the other. For instance: a library might `export default` the main class but also have named exports for utility functions.
- **What is the optional chaining operator `?.` and how does it help?** – Describe that optional chaining (ES2020) allows safe access to nested object properties [38]. For example, `user?.profile?.email` will **short-circuit** to `undefined` if any part (`user` or `user.profile`) is null/undefined, instead of throwing an error. This avoids lengthy checks like

`if(user && user.profile)`. It's very useful when dealing with data that may be incomplete or optional.

- **What is the nullish coalescing operator `??`?** – Explain that `expr1 ?? expr2` (ES2020) returns `expr1` if it's *not* `null` or `undefined`; otherwise it returns `expr2` [39]. This is similar to the `||` operator but importantly, `??` only treats `null` or `undefined` as "falsy," whereas `||` would treat other falsy values like 0 or "" as needing the fallback. Give an example: `let count = response.count ?? 0;` will use 0 only if `response.count` is null/undefined, but if `count` is 0 (a valid number), it will preserve that.
- **How do you perform module imports/exports in ES6?** – Expect something like: *"Using the `import` and `export` syntax. For example, if you have `export function doTask() { }` in a module file, you can import it in another file with `import { doTask } from './module.js';`. For default exports, you do `export default Component;` and import with `import Component from './module.js';`."* Also check understanding that ES6 modules are static (determined at compile time), which enables tree-shaking by bundlers.

Advanced

- **What are generators and what is a use case for them?** – Expand on generators (`function*` and `yield`) beyond the basics. For instance, one use case: generating an infinite sequence lazily (e.g. an ID generator) or iterating through a custom data structure. Or using generators to manage asynchronous flow (as seen in libraries like co or redux-saga). This tests deeper knowledge of ES6 generator functions.
- **What are *WeakMap* and *WeakSet* and how do they differ from Map/Set?** – Explain that WeakMaps and WeakSets (ES6) hold *weak* references to objects as keys (or values in WeakSet) [40] [41]. Because of this, those objects can be garbage-collected if there are no other references, which is not the case with a normal Map/Set (they'd prevent GC). WeakMaps/WeakSets cannot be iterated over or cleared manually – they are used for cases like caching or storing metadata about objects without preventing them from being GC'd.
- **What is a Proxy in JavaScript and what can it be used for?** – Describe the `Proxy` object (ES2015) which allows you to intercept and define custom behavior for fundamental operations on objects (like property access, assignment, enumeration, function invocation, etc.). Provide an example use case: observing or validating property accesses (e.g. implementing reactive data binding by intercepting gets/sets), or creating a simpler API like negative array indices by intercepting property access on an array.
- **How do dynamic imports work in modern JavaScript?** – Discuss that besides static `import` declarations, you can use `import()` as a function to load modules on the fly (ES2020). This returns a promise that resolves to the module. It's useful for code-splitting – e.g. loading a module only when needed (on-demand, such as loading a heavy component only when a user navigates to a certain route). Perhaps mention that this is a cornerstone of lazy-loading in bundlers and frameworks.
- **Can you name some new features in the latest ECMAScript (ES2022/ES2023/ES2024)?** – This is to test if the candidate keeps up-to-date. Possible answers: *ES2022:* class private fields and methods reached stage 4, `at()` method on arrays/strings (`arr.at(-1)`), `Object.hasOwn(obj, prop)` (a simpler hasOwnProperty), top-level `await` in modules. *ES2023:* new `Array.findLast` and `Array.findLastIndex`, `Array.toSorted` (non-mutating sort), `Map.emplace` (perhaps), etc. *ES2024:* (if known by candidate) maybe the *Record & Tuple* proposal or *Temporal* (though those might not yet be in the final spec), or the *Hashbang `#!` support* in scripts. Basically checking awareness of recent additions beyond ES6.

8. Modules and Tooling (import/export, bundlers, Webpack)

Basic

- **What is a module in JavaScript?** – Explain the concept of code modularization: a module is a file or unit of code that encapsulates functions, objects, or values and can export them for use in other modules. Prior to ES6 modules, we had patterns like IIFEs or library globals; now we use the import/export syntax to organize code into reusable pieces. Mention benefits: avoids global variables, helps manage dependencies.
- **How do you export and import functionality in ES6 modules?** – Provide a simple example: *"In module* `math.js` *you might write* `export function add(x,y){ return x+y }`. *In another file, you import it with* `import { add } from './math.js';` *and then you can use* `add(2,3)`." Also mention default export: one module can `export default` a value (e.g. a class or object), which you import without curly braces using your chosen name.
- **What is NPM and why is it useful?** – NPM (Node Package Manager) is the package registry and tool used to install third-party libraries/packages in JavaScript (especially Node and front-end with build tools) [42]. It manages dependencies of a project through a `package.json` file and allows sharing your own packages. In essence, it's how the JavaScript ecosystem shares and version-controls code libraries.
- **What is a bundler like Webpack used for?** – Explain that in modern web development, we often split our code into many modules (using ES6 import/export or other formats). A bundler like Webpack **packages** all those modules (and assets) into a few files that browsers can load. It can also transform code (via loaders/plugins – e.g. transpile TypeScript/JSX to JS, bundle CSS, etc.). The bundler helps manage dependencies and optimize the delivery (minification, code splitting, etc.).
- **What is Babel and what role does it play in modern JS development?** – Babel is a popular JavaScript compiler/transpiler. It lets developers write cutting-edge JavaScript (ES6+ or even proposals) and transpiles it down to ES5 that older environments can run. It's often used in the toolchain to ensure compatibility. For example, Babel can turn `import`/`export` and other ES2015+ syntax into code that works in older browsers, and can polyfill missing features.

Intermediate

- **What are AMD and CommonJS in the context of JavaScript modules?** – These are older module systems. AMD (Asynchronous Module Definition) was used in browsers (require.js) and loaded modules asynchronously, using a define/require syntax. CommonJS is the module format used by Node.js (using `require()` and `module.exports`) [43]. Discuss differences: AMD was browser-focused, CommonJS is synchronous and used on the server (Node). ES6 modules have largely unified module syntax in modern JS, but understanding AMD/CommonJS helps when dealing with older projects or certain build configurations.
- **What is the difference between using `require` in Node.js and `import` in ES6?** – Highlight that `require()` is CommonJS (synchronous, can be called at runtime in any order, returns `module.exports`), while `import` is ES6 (statically analyzed, must be used at the top level, asynchronous module loading behind the scenes, with `export` syntax to define what's exposed). Also mention that Node as of recent versions supports ES modules natively (using `.mjs` or package.json `"type": "module"`), but historically one had to transpile or stick to `require`. This shows knowledge of module systems interoperability.
- **What is contained in the `package.json` file and why is it important?** – Describe `package.json` as the manifest for a Node/JS project [44]. It includes metadata like the project name/version, scripts, and most importantly the dependencies and devDependencies of the project (with specific versions or version ranges). This file allows others to install the exact

dependencies via `npm install`. Also mention it can define things like the entry point (`main` field), and is used by tools to know how to build or start the project (via npm scripts).

- **What is *tree shaking* in the context of bundlers?** – Tree shaking is an optimization performed by modern bundlers (like Webpack, Rollup) to remove dead code. It leverages ES6 module static structure: if an exported function or value isn't used anywhere in the bundle, the bundler can detect that and omit it from the final build [45]. Ask for an example: if you import only one function from a utility module that has many exports, a tree-shaking bundler can drop the unused exports so they don't bloat the bundle.
- **How do bundlers achieve *code splitting* and why would you use it?** – Code splitting is the practice of splitting the bundle into multiple chunks that can be loaded on demand. Explain that bundlers allow defining split points (for example, dynamically `import()` a module, or configuration in Webpack) so that certain code is in a separate file. This is useful to lazy-load parts of the app (like a large admin panel code that the user might not need initially) to improve initial load performance. It shows understanding of optimizing large applications.

Advanced

- **Have you worked with Webpack? How do you configure it to handle different file types (like JSX, CSS, images)?** – This expects some familiarity with Webpack's concept of loaders and plugins. For instance: *"Yes, to handle JSX or ES6, you'd use Babel via the* `babel-loader`*. For CSS, you might use* `style-loader` *and* `css-loader`*. Images can use* `file-loader` *or* `url-loader` *to bundle them or embed as base64. In webpack.config.js, you configure module rules to use these loaders for different file extensions."* It's a question that checks hands-on experience with build tools.
- **What is Module Federation in Webpack 5?** – An advanced concept where multiple bundles (from potentially different builds/apps) can dynamically import code from each other at runtime, enabling micro-frontend architectures. If the candidate knows this, they might explain how Module Federation allows loading remote module code and sharing dependencies between bundles, which is useful for building applications out of separate independently deployed pieces.
- **How do ES modules work in Node.js now, and what are some challenges when mixing with CommonJS?** – Node.js supports ES modules (with `.mjs` files or `"type": "module"` in package.json). Discuss that unlike browsers, Node's ES module support had some differences (like you can't __dirname easily, need `import.meta.url`, and how to import CommonJS in ES module or vice versa). Challenges: you can't directly `require` an ES module from CommonJS without dynamic import, and default import interop differences (CommonJS default export is accessed via `.default` in ESM). This is advanced knowledge of the evolving Node module system.
- **What are source maps and how do they work in debugging?** – Explain that a source map is a file (or data embedded in a file) that maps the transformed/minified code back to the original source code. When using tools like Webpack or Babel, the output JavaScript is different from your source (due to bundling, minification, or transpilation). A source map allows dev tools to show you the original code when debugging. It contains mappings of every segment of the minified code to the original file and line. This is important for debugging production code.
- **Besides Webpack, name another build tool or bundler you have used. What are some differences?** – They might mention Rollup (great for libraries, does tree-shaking naturally), Parcel (zero-config bundler), Browserify (an older bundler), or newer ones like Vite/Esbuild (which use faster compilers under the hood). They should highlight differences like configuration complexity, performance, use cases (e.g., Rollup for libraries, Webpack for apps, Vite for dev speed with ES modules, etc.). This assesses awareness of the tooling landscape.

9. Error Handling and Debugging

Basic

- **How do you handle errors in JavaScript code?** – Discuss the `try...catch` statement for catching exceptions in synchronous code. For example: *"You wrap code that might throw an error in a try block, and handle errors in the catch block."* Mention the `finally` clause that runs regardless of an error (often for cleanup). This tests knowledge of the fundamental error handling mechanism.
- **What happens if an error is thrown inside a promise or async function?** – Explain that in a Promise, a thrown error (or a rejected promise) will call the `.catch` handler. In an `async` function, throwing an error causes the returned promise to reject, which should be handled with try/catch around the await or a `.catch` on the promise. This shows understanding of error propagation in async contexts.
- **What tools do you have for debugging JavaScript in the browser?** – Expected answer: the browser developer tools (DevTools). For example, using the **Sources** panel to set breakpoints, step through code, inspect values; the **Console** for logging output with `console.log()` as a quick debugging technique [46] ; perhaps the **Network** panel to debug requests. Basically ensure the candidate knows how to use breakpoints and the debugger in a modern browser.
- **What does `console.log` do, and what are some other console methods?** – `console.log` prints messages to the browser's console (or Node console). Other useful ones: `console.error` (prints in red / error stack), `console.table` (formats array/objects as a table), `console.trace()` (prints a stack trace), `console.time/timeEnd` (for simple timing). This question checks familiarity with basic debugging output beyond just `alert()` or simple logs.
- **How can the `debugger` statement be used in JavaScript?** – Explain that placing the `debugger;` statement in code will cause the JS execution to pause and open the debugging tools (if they're open, it will break there). It's essentially like setting a breakpoint in code without using the UI. It's useful for hard-to-reproduce issues where you want to ensure execution stops at a certain point.

Intermediate

- **What is the call stack, and how is it useful in debugging?** – Describe the call stack as the structure that keeps track of function calls. In debugging, when an error is thrown, the browser (or Node) often provides a stack trace – a list of function calls that led to the error. Understanding the call stack helps you trace the sequence of calls. Mention that you can inspect the call stack in DevTools when paused on a breakpoint to see how you got to that code.
- **How would you debug an undefined value or unexpected NaN in your program?** – Look for a systematic approach: using `console.log` or breakpoints to inspect variable values at certain points, checking function arguments, verifying the order of operations. Essentially, the candidate should describe narrowing down the source of the undefined or NaN by tracing through the code path. This tests practical debugging thought process.
- **How do you handle errors in asynchronous code (like in a callback or promise)?** – For callbacks, you typically check for an error argument (Node.js callback pattern: `function(err, result) { if(err) { /* handle */ } }`). For promises, use `.catch` or try/catch in an `async` function. Make sure to mention that a `try...catch` won't catch errors inside async callbacks without proper handling. This ensures they understand that error handling needs to be done differently for async operations.
- **What is a stack trace?** – Explain that a stack trace is the list of function calls that were active when an error occurred (or when `console.trace()` is called). It usually shows the function

names and line numbers for each step in the call stack. When an error is thrown, reading the stack trace helps pinpoint where the issue originated and how the code got there. Perhaps mention how to read one (the top is the location of error, the next line is the caller, etc.).
- **What strategies or tools do you use to debug memory leaks or performance issues?** – This could mention the memory profiler in Chrome DevTools (to take heap snapshots and compare, or observe timeline for memory growing). For performance, using the Performance tab to record slow frames or high CPU usage. The answer might include using `console.profile()` or investigating suspicious object retention. While not every developer has done this, a strong candidate might mention observing DOM nodes that aren't being garbage collected or using browser task manager, etc.

Advanced

- **How can you catch global errors in a web page or Node.js app?** – In the browser, mention the global `window.onerror` event or using `window.addEventListener('error', ...)` for runtime errors, and `unhandledrejection` for unhandled promise rejections. In Node.js, mention catching `process.on('uncaughtException')` or `process.on('unhandledRejection')` (though using those is usually a last resort) [47] [48]. Emphasize these are for logging or last-resort measures, not for normal control flow.
- **What is the purpose of source maps in debugging, and how do you use them?** – As discussed earlier, source maps map minified/transpiled code back to the original. The devtools use them so you can place breakpoints in your original source files and step through TypeScript or ES6+ code even if the browser is running compiled ES5. Explain how you would enable or generate source maps (often an option in build tools, like `devtool: 'source-map'` in Webpack). This is advanced in the sense of build pipeline knowledge.
- **Explain a situation where you had a bug that was hard to find – how did you debug it?** – This is open-ended to see the candidate's personal experience with debugging. A strong answer will outline a specific scenario and the systematic approach taken (e.g. binary search through code, adding logs, simplifying the problem, etc.). It reveals problem-solving and persistence in debugging complex issues.
- **What is *linter* (like ESLint) and how can it help prevent errors?** – Discuss that a linter is a static analysis tool that checks code for potential errors or style violations without running it. ESLint, for example, can catch things like unused variables, undeclared variables, improper comparisons, etc., which might prevent certain bugs. It enforces best practices that can reduce bugs (for instance, warning if you use a variable that was never defined – which could prevent a ReferenceError at runtime). This touches on tooling that improves code quality.
- **How do you debug Node.js applications?** – Look for mentions of the Node inspector (`node --inspect` and using Chrome DevTools or VSCode to attach) or simply using `console.log` in Node. They might mention debugger statements in Node or using dedicated debugging tools like `ndb` or VSCode's debug panel. It's advanced in the sense that not all front-end devs debug Node, but full-stack ones should know.

10. Frontend Frameworks (React, Angular, Vue)

Basic

- **What is a Single Page Application (SPA)?** – Define an SPA as a web application that loads a single HTML page and dynamically updates content without full page reloads as the user interacts [49]. Explain how SPAs use client-side routing and AJAX/fetch calls to update the view. Mention one advantage (smooth user experience, like a desktop app) and one challenge (needs explicit handling for SEO or initial load performance).

- **What is a component in front-end frameworks?** – Explain the concept of a UI component: a self-contained piece of the interface (often with its own logic and styling) that can be reused. In React, a component is typically a function or class that returns JSX. In Angular, a component is a class with a decorator defining a template. In Vue, it's an options object or class. The key idea: components encapsulate structure (HTML), style, and behavior for a piece of UI.
- **Why would you use a framework like React or Angular instead of vanilla JS?** – Look for understanding of problems frameworks solve: managing complex UI state, updating the DOM efficiently (Virtual DOM diffing in React [50]), handling routing, and offering abstractions for data flow (one-way data binding in React/Redux vs two-way in Angular). They might mention that frameworks provide structure, enforce patterns, and have a community/ecosystem of ready solutions, which can speed up development for large apps.
- **What is React?** – A basic description: React is a JavaScript library (from Facebook) for building user interfaces. It uses a declarative style where you describe what the UI should look like given some state, and it handles rendering and updating the DOM (using a virtual DOM diff). Look for mention of components and JSX. This is straightforward, but an interviewer wants to ensure the fundamentals are understood.
- **What is the Virtual DOM and how does React use it?** – Explain that the Virtual DOM is an in-memory representation of the real DOM [51]. React uses it to batch and optimize updates: when state changes, React computes a new virtual DOM and diffs it with the previous one to figure out the minimal actual DOM mutations needed. This improves performance by avoiding unnecessary DOM manipulations and reflows. It's a key concept underlying React's efficiency.

Intermediate

- **How does two-way data binding differ from one-way data flow?** – Define two-way binding (used in Angular, also in Vue with `v-model`) as a mechanism where UI fields (inputs) and model data automatically keep in sync – updating one immediately updates the other. One-way data flow (used in React) means data flows in one direction: from parent to child via props, and child communicates changes via events/callbacks (or state is lifted up). Discuss the trade-offs: two-way is convenient for forms, but one-way (unidirectional) can make data flow easier to reason about in complex apps.
- **What are React Hooks and why were they introduced?** – Explain that Hooks (like `useState`, `useEffect` introduced in React 16.8) let you use state and side-effects in functional components, replacing many uses of class components. They were introduced to simplify code reuse (no need for HOC/render props for stateful logic) and to make the component lifecycle more linear (the `useEffect` hook can handle side effects that were previously in lifecycle methods). Mention a couple common hooks: `useState` for state, `useEffect` for side effects, `useContext` for context, etc.
- **What are the key differences between React and Angular?** – Expect an answer comparing: React is a library focused on the view (you often use additional libraries for routing, state management), it's based on a virtual DOM and JSX. Angular is a full-fledged framework (from Google) with opinionated structure (MVC-ish), two-way binding, built-in dependency injection, uses TypeScript by default, and template HTML with directives. Also, Angular updates the DOM via its digest cycle and zone.js for change detection, whereas React explicitly re-renders on state change. A candidate might also mention Angular is heavier and has a steeper learning curve, while React's ecosystem gives more flexibility (but also more decisions to make).
- **How do you manage state in a React application?** – They should talk about local component state vs global state. For local, use `useState` or class `setState`. For global or shared state, mention patterns or libraries: lifting state up to common ancestors, using React Context for simple cases, or external state management libraries like Redux or MobX for more complex needs. They might also mention the newer hooks like `useReducer` for complex local state, or

Redux's context-based hooks. If the role expects knowledge of one, they might specifically expect "Redux" as an answer for global state.

- **What is the difference between a class component and a functional component in React?** – Class components can have state and lifecycle methods (pre-Hooks), and you need to use `this`. Functional components were originally just presentational (no internal state or lifecycle aside from props) but now with Hooks they can also manage state and side effects. Functional components are generally shorter and encourage simpler, more readable code. They also avoid some pitfalls (`this` binding, etc.). Since React's Hooks, functional components are the norm. Also mention performance: historically, there were minor differences, but now either can be optimized (with `React.memo` or `PureComponent`).

Advanced

- **What is server-side rendering (SSR) in the context of SPAs and why use it?** – Explain SSR as rendering the React/Angular/Vue components to HTML on the server (Node.js) before sending it to the client, rather than the usual pure client-side rendering. Benefits: faster first paint and better SEO because the content is present on initial load. There are costs: more complex build setup, potential hydration issues. For React, mention frameworks like Next.js that facilitate SSR. This question checks knowledge of performance optimization and SEO concerns in frameworks.
- **What is Redux and how does it fit into a React application?** – Redux is a state management library that uses a single immutable state tree and pure reducer functions to manage state changes (with actions dispatched to modify state). In a React app, Redux can hold global app state and React components subscribe to the parts of the state they need. Explain key concepts: store, actions, reducers, dispatch. Also mention one benefit: predictable state transitions and easier debugging (with Redux DevTools time-travel). Possibly mention alternatives or that Redux is one of many options (MobX, Context API, etc.), showing awareness of the ecosystem.
- **How does Angular use dependency injection, and why is it useful?** – Angular's dependency injection system provides instances of services to classes (like components) by looking at constructor parameters (or @Injectable metadata) [47] [48]. This allows you to easily swap out implementations for testing (mock services) and promotes modularity. The candidate should explain that in Angular, if a component needs a service, you declare it in the constructor and Angular's DI framework provides it. This is something React/Vue don't have built-in (they use context or just imports), so it's a distinguishing feature of Angular.
- **Explain the concept of the *virtual DOM diffing* or reconciliation algorithm in React.** – This is advanced: React's reconciliation algorithm determines how to update the real DOM efficiently [52]. The candidate might mention heuristics like assuming elements of the same type can be updated, using keys to track list item identity to minimize re-renders, etc. A strong answer might reference that React diffing is O(n) for tree updates because it doesn't do an exhaustive comparison (it won't re-order nodes unless keys indicate movement). This tests deep knowledge of React's internals.
- **What are some best practices for optimizing performance in a large React (or Vue/Angular) application?** – Look for answers like: using production builds (minified, no dev checks), shouldComponentUpdate (React PureComponent or memo) to prevent unnecessary re-renders, code-splitting large bundles, using Web Workers for expensive calculations, debouncing/throttling events that trigger heavy updates (like window resize or scroll handlers). In Angular, one might mention onPush change detection strategy to optimize checks, or trackBy for *ngFor to optimize list diffing. For Vue, maybe mention computed properties caching and immutability/watchers considerations. This gauges practical performance tuning knowledge beyond basics.

11. Backend JavaScript (Node.js, Express.js)

Basic

- **What is Node.js and what is it used for?** – Node.js is a runtime environment that allows JavaScript to run on the server (outside of a browser) [53] . It uses Google's V8 engine under the hood. Describe use cases: building web servers or network tools, CLI scripts, real-time applications (like chat servers). Emphasize non-blocking, event-driven architecture which makes it good for I/O heavy tasks (not as good for CPU-heavy tasks unless using workers).
- **How is JavaScript in Node.js different from JavaScript in the browser?** – Mention that core language is the same, but the environment provides different APIs. In Node, you don't have `window` or the DOM, but you have built-in modules like `fs` for file system, `http` for servers, etc. Also mention module system difference: Node historically uses CommonJS (`require`) whereas browsers use ES modules (though Node now supports ESM too). And global object is `global` in Node vs `window` in browser.
- **What is Express.js?** – Express is a minimalist web application framework for Node.js [54] . It provides abstractions for creating servers, defining routes (URL endpoints) and middleware for handling HTTP requests and responses easily. Basically, it simplifies building API endpoints or web applications by handling low-level details like raw HTTP requests, so you can define "when GET /users happens, run this function." Mention that it's unopinionated, allowing use of various middleware for things like parsing JSON, handling cookies, etc.
- **How do you create a simple HTTP server in Node?** – Two ways: using Node's built-in `http` module (e.g. `http.createServer((req, res) => { res.end('Hello'); }).listen(3000)`) which is fairly low-level, or using Express (`const app = express(); app.get('/', (req,res)=>res.send('Hi'); app.listen(3000);`) which is simpler. The expectation is that the candidate knows at least one approach. This indicates familiarity with basic Node API usage.
- **What are some common built-in modules in Node.js?** – For example, `fs` (file system), `path` (utilities for file paths), `http/https` (for creating servers or making requests), `os` (info about the operating system), `events` (EventEmitter class), `util`, etc. Listing a few shows familiarity. Might also mention that Node's standard library provides these to avoid needing external libraries for common tasks.

Intermediate

- **What is event-driven architecture in Node.js?** – Node's design is event-driven: operations (especially I/O) emit events (or call callbacks) when complete, rather than blocking. The **EventEmitter** class is central – many core modules inherit from it (e.g. streams). For example, a TCP server emits a "connection" event when a client connects. Understanding: Node handles a lot via events and callbacks instead of linear execution, which is why it can handle many concurrent connections efficiently.
- **Explain the non-blocking I/O model of Node.js.** – This is a chance to describe how Node uses the event loop (libuv under the hood) to handle I/O asynchronously [55] [56] . When a file or network request is made, Node will initiate the operation and then continue running other code – when the I/O operation finishes, a callback is queued to handle the result. Thus, a single thread can manage many I/O operations without waiting on each. It's important to mention that this is great for I/O-bound tasks, but CPU-bound tasks can still block the single thread (unless using worker threads).
- **What is middleware in Express.js?** – Middleware are functions that have access to the request and response objects and a `next` function. They can modify the request/response or end the response, or call `next()` to pass control to the next middleware. They're used for tasks like

logging, authentication checks, parsing request bodies, error handling, etc. In Express, you might say "It's the layers of processing on an incoming request – for example, one middleware might parse JSON, another might check a user token, another might handle the route logic."

- **How do you connect Node.js to a database (e.g. MongoDB or MySQL)?** – Not expecting actual code, but knowledge of approach: using packages like `mongodb` or an ORM like Mongoose for MongoDB, or `mysql2` / `pg` for SQL, or higher-level ORMs like Sequelize/TypeORM/Prisma. You typically use a client library that provides functions to connect (often asynchronously) and then you can perform queries. Possibly mention using environment variables for DB credentials and the need to handle async operations (promises/callbacks) when querying a database in Node.
- **What is the purpose of the `package.json` file in a Node project?** – Reiterate that it lists the project's dependencies and scripts [44]. It contains metadata (name, version), but importantly it tracks dependencies with specific versions so that running `npm install` on another machine installs the same packages. It also can define build or start scripts (like `"start": "node index.js"`). Basically, Node projects rely on package.json to manage their module dependencies and project info.

Advanced

- **What is clustering in Node.js and why might you use it?** – Explain that because Node is single-threaded, clustering is a technique to spawn multiple Node processes (workers) to take advantage of multi-core servers. The `cluster` module or process managers (PM2) can distribute incoming requests across multiple worker processes. This improves throughput by parallelizing the work across CPU cores, albeit with separate memory spaces. Essentially, "Cluster = running multiple instances of the server for better performance on multi-core machines."
- **Explain the role of the EventEmitter class in Node.** – The EventEmitter is a core pattern in Node (found in the `events` module) [57]. It allows objects to emit named events and have listeners for those events. Many Node built-ins use it (e.g. streams: a readable stream emits 'data' events). As an answer, mention how you can create your own EventEmitter to decouple logic (observer pattern). Possibly include a quick example: *"const emitter = new EventEmitter(); emitter.on('ready', () => console.log('Ready!')); emitter.emit('ready');"*.
- **What are streams in Node.js?** – Define streams as objects that allow reading/writing data piece by piece (chunks) rather than all at once [58]. Node has readable, writable, duplex (both), and transform streams. They're used for handling large data efficiently (like reading a big file line by line, or sending an HTTP response chunk by chunk). Provide a use case: reading a file using a stream uses constant memory, whereas reading it fully would use memory proportional to file size. Also mention piping: you can pipe streams to each other (like reading from a file stream and piping to a response, which streams it to the client).
- **How does Node.js handle child processes, and what is the difference between `spawn` and `fork`?** – Node can launch child processes using the `child_process` module. `spawn` launches a new process with a given command, returning streams for stdout/stderr. `fork` is a special case of spawn that runs a new Node V8 instance and sets up IPC (communication channel) between parent and child [59]. Essentially, `child_process.fork` is used to spawn another Node script as a child that can send messages back and forth with `process.send()`. This is advanced, but tests understanding of scaling beyond single process (related to cluster, where fork is used to create worker processes).
- **How do you handle security concerns in a Node/Express application?** – Looking for points like: validating and sanitizing input to prevent injection attacks (SQL injection, etc.), using HTTPS and proper headers, preventing XSS by escaping output (though XSS is more a front-end concern, but templates can be involved), using libraries like Helmet (to set security-related HTTP headers) [60], enabling CORS properly (with the `cors` middleware) if needed, storing secrets (like DB passwords or JWT secret) securely (not in code – use env variables). Also mention

authentication best practices (storing passwords hashed, using cookies vs JWT securely – HttpOnly cookies to mitigate XSS, proper CORS settings to mitigate CSRF). This shows a broad awareness of web security in the Node context.

12. Testing and Tooling (Jest, Mocha, ESLint)

Basic

- **Why is unit testing important?** – Explain that unit tests verify the correctness of individual units of code (usually functions or modules) in isolation. They help catch bugs early, document expected behavior, and give confidence when refactoring (regressions). It's an essential practice for reliable software. Possibly mention other types of tests (integration, end-to-end) to show context, but focus on unit tests.
- **What is Jest and what is it used for?** – Jest is a popular JavaScript testing framework [61], originally from Facebook, often used with React but not limited to it. It provides a test runner, assertion library, and features like snapshot testing and coverage out of the box. Essentially, with Jest you can write tests (`test()` or `it()` blocks) and run them to see if your code behaves as expected. It's known for its simplicity and built-in mocking abilities.
- **How do you run tests in a typical JavaScript project?** – Usually via a script in `package.json` like `"test": "jest"` or using a test runner CLI. So answer: *"Often by running* `npm test` *or a similar command which invokes a test runner (like Jest or Mocha). The runner finds test files (maybe named* `.spec.js` *or* `.test.js` *) and executes them, reporting passing or failing tests."* This checks if they have actually run tests, not just know theory.
- **What's the difference between a unit test and an integration test?** – Unit tests focus on small units of code in isolation (e.g., a single function, mocking any dependencies). Integration tests cover the interaction between components or modules – for example, testing a whole API endpoint which involves database, or a series of functions working together. Integration tests are usually closer to the real environment (less mocking), and can catch issues in how units integrate. It's important to know because strategy for testing often involves both.
- **What is ESLint and why would you use it?** – ESLint is a linter for JavaScript. It analyzes code for stylistic consistency and common errors. It can catch things like unused variables, undefined variables, improper usage of equality, and can enforce coding style (quotes, spacing, etc.). Using ESLint helps maintain code quality and can prevent certain types of bugs (for example, it can warn if you used a variable that was never declared, which could avoid a ReferenceError) before running the code. It's a development tool, not used at runtime.

Intermediate

- **How do you test asynchronous code with Jest (or Mocha)?** – The answer should include methods like returning a promise from the test, using async/await in the test, or using done callbacks (in Mocha or Jest for older style). For example: *"In Jest, you can make the test function* `async` *and use* `await` *inside it, or return the promise directly. Jest will wait for the promise to resolve. Alternatively, for callback-based async, you can use the* `done` *callback and call* `done()` *when finished."* This tests knowledge of handling async in tests properly.
- **What is mocking in the context of testing?** – Mocking is faking parts of the system that the unit under test interacts with. For example, if a function makes a network request, in a unit test you would mock the network module so it doesn't actually call the network but simulates a response. Mention Jest's ability to create mocks/spies (`jest.fn()` or `jest.mock(module)`) or in general how you might use dummy implementations in tests. The idea is to isolate the unit and also make tests run faster and not depend on external systems.

- **What is a test *suite* and a test *case*?** – In frameworks like Mocha/Jest: a test suite is a collection of related test cases, often defined by a `describe` block [62]. A test case is an individual test, defined by an `it` or `test` function, which contains assertions. This is basic terminology but important: suite (group of tests, often for one module or feature) vs test case (a specific scenario to verify).
- **How do you measure code coverage and why might it be useful?** – Code coverage is measured by running your test suite and instrumenting the code to see which lines/branches were executed. Tools like Istanbul (integrated in Jest) generate a coverage report (often seen as percentages for statements/branches/functions covered). It's useful to identify untested parts of your code. A candidate should mention that while 100% coverage doesn't guarantee no bugs, low coverage might indicate not enough tests. It's a metric to guide improving test completeness.
- **Have you used any assertion libraries or BDD style testing frameworks?** – Potential answer: *"Yes, testing frameworks usually come with assertion libraries. For example, Jest has built-in matchers like* `expect(x).toBe(y)`*. In Mocha, you might use Chai for assertions (expect/should style). BDD style refers to writing tests in a behavior-descriptive way (using* `describe/it` *or* `given/when/then`*)."* They might also mention Jasmine (similar to Jest) or other tools. This question is to see familiarity with the ecosystem beyond just one tool.

Advanced

- **What is snapshot testing in Jest and when would you use it?** – Snapshot testing involves taking a "snapshot" of a rendered component's output (or any serializable value) and saving it to a file. Future test runs compare the output to the saved snapshot [61]. It's commonly used in React to ensure UI components don't unexpectedly change. A good answer: *"In Jest, you can use* `expect(tree).toMatchSnapshot()`*. The first run saves the output (e.g., React render output) to a snapshot file. Subsequent runs will fail if the output differs. It's useful to detect unintended changes in UI or serialized output. However, you have to update snapshots intentionally when changes are valid."*
- **How can you test a React component?** – There are a few approaches: using React Testing Library (preferred modern approach) which focuses on component output and user interactions (DOM-based testing), or using Enzyme (older, more implementation-driven testing of React components). They should mention simulating events, asserting that certain text or elements appear given props/state. Essentially, test components by rendering them (perhaps with a testing renderer or in a jsdom environment) and making assertions on the rendered output or behavior (like clicking a button changes something).
- **What is end-to-end (E2E) testing and which tools have you used for it?** – E2E tests simulate user interactions in a full application environment (often a headless browser). Tools: Cypress, Selenium (or WebDriverIO, Puppeteer, Playwright). The candidate might say: *"E2E tests launch the whole app (for web, maybe a local dev server) and then script a browser to perform actions like a real user and verify the results (like seeing certain text on the page). I've used Cypress which is great for this – it runs in the browser and provides a nice API to simulate clicks and check DOM."* This checks experience beyond unit testing, into integration/system testing.
- **What is TDD (Test-Driven Development)?** – Describe TDD as a development process where you write tests for a new feature or function *before* writing the implementation. The cycle is red-green-refactor: write a test (it fails because implementation isn't there yet), implement the minimal code to make it pass, then refactor if needed, ensuring tests still pass. Ask if they have used it and what the benefits are (forces thinking about requirements/API design upfront, leads to better test coverage, etc.). This is more methodology, but good to see if they understand it.
- **How does continuous integration (CI) relate to testing?** – They should mention that CI is setting up an automated process (on a service like GitHub Actions, Jenkins, Travis CI, etc.) to run

your test suite (and other checks) on each commit or pull request. This ensures that new changes don't break existing functionality – the tests act as a safety net. Possibly mention that CI can also run linters, build the project, and even deploy if all tests pass (CD). This demonstrates understanding of how testing fits into the DevOps pipeline, which is valuable knowledge for higher-level roles.

13. Performance Optimization and Memory Management

Basic

- **What are some ways to make a web page load faster?** – Expected answers: Minify and bundle resources (JavaScript/CSS) to reduce download size [63], use caching (browser cache or server-side cache, set appropriate cache headers), compress assets (gzip or Brotli), defer loading of non-critical resources (like using async/defer for scripts, lazy loading images), reduce the number of HTTP requests (combine files, use sprites or inline images for small icons, etc.). This tests general awareness of front-end performance best practices.
- **What is a memory leak in a JavaScript application?** – Define a memory leak as a scenario where an application keeps holding references to objects that are no longer needed, thus preventing the garbage collector from reclaiming that memory. Give a simple example: if you attach event listeners to DOM elements and never remove them even after those elements are removed, the listener might hold a reference preventing GC of those elements. Over time, this causes increasing memory usage.
- **How does JavaScript's garbage collector generally work?** – Without going too low-level, mention mark-and-sweep: the GC will mark objects that are reachable (through the global scope or current execution contexts) and assume anything not marked is garbage to collect. JavaScript automatically frees memory for objects that are no longer referenced (you typically can't explicitly free memory). Emphasize that developers usually don't manage memory directly, but need to avoid patterns that inadvertently keep references (causing leaks).
- **What is the difference between `requestAnimationFrame` and `setTimeout` for animations?** – `requestAnimationFrame` is a browser API that calls a callback right before the next repaint, which is usually ~60 times a second (optimized for the screen's refresh rate). It's more efficient for animations than `setTimeout(..., 16)` because it can sync with the browser's internal rendering and pause when the tab is not visible, etc. This shows understanding of performance-friendly animation techniques.
- **Why is manipulating the DOM often a performance bottleneck?** – Explain that updating the DOM can trigger reflows (layout) and repaints in the browser, which are expensive if done too frequently or on large portions of the page. For example, changing styles or adding elements repeatedly in a loop causes multiple layouts. Best practice is to batch DOM changes or use techniques like document fragments or frameworks that minimize direct DOM access. Essentially: DOM is slow relative to JS in memory, so doing lots of DOM ops can hurt performance.

Intermediate

- **What are debouncing and throttling?** – Debouncing: ensure a function is not called again until a certain time has elapsed since the last call. Useful for events that fire rapidly (e.g. window resize, keyup) – you only act when the user has stopped triggering the event for, say, 300ms. Throttling: ensure a function is called at most once in a specified period (e.g. at most once every 100ms), ignoring extra calls in between. This is useful to limit event handling frequency (like on scroll events) [64]. Ask for practical uses and maybe how to implement (with timers).

- **How would you find a memory leak in a large single-page application?** – They should mention the Chrome DevTools memory panel: taking heap snapshots and comparing them over time (see what objects keep increasing), using the Timeline (Performance) to record and see if memory is ever freed, using tools to find detached DOM nodes that aren't collected (a common leak). Also mention checking event listeners that are not removed. Essentially, outline a process: reproduce the scenario of leak (e.g., navigate around or perform actions repeatedly), observe memory usage, use profiling tools to pinpoint what grows or what isn't collected.
- **What are web workers and when might you use one?** – Web workers allow running JavaScript on a background thread separate from the main UI thread. You'd use them for heavy computations or operations that would otherwise block the UI (like processing a large data set, image processing, etc.) so that the main thread remains responsive. For example, if you need to sort a huge array or crunch numbers, offload to a worker and communicate results back via message passing. The trade-off is you can't access the DOM from a worker, and there's overhead in messaging.
- **Explain how caching can improve performance in a web application.** – Different levels: Browser caching (if resources have appropriate headers, the browser can reuse cached files instead of re-fetching them on subsequent visits). Application-level caching: storing results of expensive computations or AJAX responses (maybe in memory or IndexedDB) so you don't recompute or re-fetch frequently. CDN caching: using a Content Delivery Network or proxy to cache static assets geographically closer to users. They should demonstrate understanding that hitting a cached response is faster than recomputing/refetching every time.
- **What is the performance impact of JavaScript on initial page load, and how can you mitigate it?** – The answer might cover things like: if you send too much JS to the client, parsing and executing it can slow down initial load. Mitigations: code splitting (so initial bundle is smaller, load other scripts later), deferring non-critical scripts (use `async` / `defer` or load scripts after content is rendered), minification to reduce size, using tree shaking to drop dead code, and possibly using techniques like server-side rendering or static site generation so the user can see something before all JS loads. This shows awareness of how JS bundling and loading affects performance.

Advanced

- **What are some common causes of memory leaks in Node.js or front-end, and how do you fix them?** – Causes: Holding references in closures long after they're needed (e.g., a large object in a closure that outlives its usefulness), not destroying timers or intervals (they keep callbacks referenced), global variables that accumulate data, for front-end: detached DOM nodes as mentioned, or large caches that grow indefinitely. Fixes: manually null out references if needed (rarely, but in complex structures maybe), properly remove event listeners or intervals on teardown, avoid global state that piles up, use profiling tools to find leaks. In Node, memory leaks might come from not releasing resources (though GC usually handles most, leaks often are due to logic keeping unwanted data).
- **Explain the term "layout thrashing" (or Forced Synchronous Layout) and how to avoid it.** – Layout thrashing happens when you alternately measure and mutate the DOM in a way that forces repeated reflows. For example: in a loop you do `element.clientHeight` (browser calculates layout) then `element.style.height = ...` (invalidate layout), then measure again, etc. To avoid: batch all reads (measurements) first, then do writes (mutations) together, or use techniques like `documentFragment` for multiple DOM insertions, or simply minimize accessing layout properties in tight loops. This is a specific performance pitfall advanced devs should know.
- **How does React (or another framework) optimize updates to the DOM?** – For React, mention the virtual DOM diffing and batching of state updates (e.g., setState in event handlers is

batched). For Angular, mention zones and change detection strategies (like OnPush to reduce checks). For Vue, mention its reactivity system and virtual DOM as well. Essentially, how frameworks reduce actual DOM operations: by computing minimal diffs (React/Vue) or by checking data change references (Angular OnPush, etc.). This shows in-depth knowledge of framework performance.

- **What is tail call optimization and does JavaScript support it?** – ES2015 formally added tail call optimization in the spec (in strict mode, a tail recursive function could be optimized to not grow the stack). In practice, as of now, only Safari had implemented proper TCO. Others (V8) do not, meaning writing recursive functions might still blow the stack for large inputs. This is a very advanced topic; mentioning it shows knowledge of language implementation details. If the candidate knows it: *"TCO is when the last action of a function is calling another function (or itself), the engine can reuse the current stack frame instead of adding a new one, to save memory. The JS spec allows it (in strict mode), but currently support is inconsistent."*
- **When profiling a web app, you notice a lot of "Recalculate Style" and "Layout" entries in the timeline. What might be the cause?** – This indicates frequent style changes or DOM changes causing reflows. Possibly the code is adding/removing classes or DOM nodes too often, or maybe doing complex layout (like setting many elements' styles individually). The cause could be too much DOM manipulation within an animation frame or something like that. The answer should be: *"It suggests our code is triggering layout calculations repeatedly, perhaps via style changes or reading layout info (like calling* `offsetHeight` *) in a loop. To fix it, we'd batch changes or reduce those measures."* This tests familiarity with Chrome DevTools performance timeline and interpreting it.

14. Security (XSS, CSRF, CORS, etc.)

Basic

- **What is Cross-Site Scripting (XSS)?** – XSS is a vulnerability where an attacker injects malicious JavaScript into a website, which then runs in other users' browsers [65] . Explain the two common types: reflected (immediate, in a response based on user input like query param) and stored (attacker's script is stored on the server, e.g. in a database comment, and served to victims). Consequences: cookie theft, site defacement, redirecting users, etc. Prevention: sanitizing user input and output encoding (escape HTML so script tags don't execute), using Content Security Policy, HTTPOnly cookies to protect session tokens, etc.
- **What is CSRF (Cross-Site Request Forgery)?** – CSRF tricks a user's browser into making a request (that includes their credentials like cookies) to a vulnerable site, without their intention [48] . For example, an attacker has you click a link or visit a page that issues a state-changing request (transfer money, etc.) on another site where you're logged in. Prevention: include CSRF tokens in forms/requests (the attacker's site can't get that token), use SameSite cookies (to block sending cookies on cross-site requests) [66] , or ensure critical requests require things that can't be made cross-site (like custom headers which aren't sent by <img> or forms).
- **What is CORS and why do we need it?** – CORS (Cross-Origin Resource Sharing) is a browser security mechanism that prevents scripts from making requests to a different domain than the one that served the page, unless that target domain allows it via specific headers [67] . Explain that by default, a web page can only request resources from the same origin (same scheme/domain/port). If you need to call an API on another domain, that server must send an `Access-Control-Allow-Origin` header permitting it. So CORS is essentially a controlled relaxation of the Same-Origin Policy to allow safe cross-domain communication. The question expects at least mention of the header and that it's enforced by browsers (and that it's about security to prevent malicious cross-site requests).

- **What is the Same-Origin Policy?** – It's the security policy in browsers that restricts how documents or scripts loaded from one origin can interact with resources from another origin [68]. Essentially, without same-origin policy, any site could read data from any other site's pages you have open (which would be a huge security issue). It means e.g. JavaScript on `evil.com` cannot read data from `bank.com` (like your account info) just because you have it open. Only if the origins match or policies like CORS allow, can cross-origin interactions happen. This underpins why CORS exists.
- **What are HTTP-only cookies and how do they relate to security?** – An HTTP-only cookie has a flag that makes it inaccessible to JavaScript (document.cookie) [69]. It's only sent in HTTP requests by the browser. This helps mitigate XSS attacks from stealing session cookies, because even if an attacker injects JS, that JS cannot read the cookie value (if it's HTTP-only). So you typically mark session cookies as HttpOnly so that a successful XSS doesn't immediately compromise a user's session (it could still do other things though, like perform actions as the user, but not directly grab the token).

Intermediate

- **What are the main differences between XSS and CSRF?** – XSS is injecting malicious script into a site; it requires finding a place where user input is not properly escaped (a vulnerability on the site itself) [70]. CSRF is tricking a user's browser to make unintended requests; it exploits the fact that browsers automatically include credentials (like cookies) in requests to the target site. So XSS is about injection and running code in victim's browser, while CSRF is about forging requests from the victim's browser. Mitigations differ: XSS – sanitize output, CSP, etc., CSRF – tokens, SameSite, etc.
- **How does the SameSite cookie attribute help protect against CSRF?** – SameSite can be set to `Lax` or `Strict` to control whether cookies are sent on cross-site requests [71]. If set to Strict, cookies (like session cookies) won't be sent when navigating from another domain (like clicking a link from attacker.com). Lax allows cookies on top-level navigation GETs (so your login cookie still works if you click a link to the site) but not on other cross-site requests like POST forms or iframes. This reduces CSRF vectors because the malicious site cannot cause the user's browser to include the session cookie in a cross-site POST (if using Strict or Lax appropriately).
- **What is Content Security Policy (CSP)?** – CSP is a security header that allows site owners to specify which sources of content are trusted (scripts, images, styles, etc.). For example, a CSP can say "only execute scripts from mydomain.com or trusted-cdn.com". This helps mitigate XSS because even if an attacker injects a `<script>` tag or tries to load an external script, the browser will block it if it violates the CSP [72]. It can also block eval() and inline scripts if configured. Essentially it's defense-in-depth against XSS by limiting what content the browser can load/execute.
- **How do you prevent SQL injection in a Node/Express application?** – Use parameterized queries or ORM query builders that escape inputs properly, rather than building query strings by concatenation. E.g., with node's MySQL library or PostgreSQL, use `?` or `$1` placeholders, which ensure user input is treated as data, not code. Additionally, ORMs usually handle this automatically. (SQL injection might not be explicitly in the category list, but it's a common security question especially for backend Node.)
- **What is a HTTP security header you always set in your applications?** – Could mention any: Strict-Transport-Security (HSTS) to force HTTPS, X-Frame-Options to prevent clickjacking (e.g. `DENY` to prevent site from being framed), X-XSS-Protection (though modern browsers have mostly superseded it or it's on by default), Content-Security-Policy as discussed, X-Content-Type-Options: nosniff (to prevent MIME type sniffing), etc. This shows familiarity with common security hardening via headers.

Advanced

- **What is a timing attack and can it affect JavaScript code?** – Timing attack is when an attacker gains information by measuring how long certain operations take. In web, this might be outside typical JS, but e.g. an attacker could measure response times to guess if a username exists (the server might respond faster when user is not found vs hashing a password when user is found), or in JS context, maybe a function that early-exits could leak information via performance differences. Mitigation can involve making operations take constant time if possible for sensitive comparisons (like not using naive string compare for secrets). For front-end, maybe not as relevant except maybe side-channel via Web Workers or such.
- **What is prototype pollution in JavaScript?** – Prototype pollution is a type of attack where an attacker manipulates the default object prototype (Object.prototype) or other prototypes by injecting properties (often via merging objects or parsing JSON) [73] . If code merges user input into an object without proper checks, someone could do `__proto__` in their JSON and add properties to Object.prototype, which might then appear everywhere (and could be exploited to change application behavior). Prevention: avoid using unsanitized objects for deep merges, use `Object.create(null)` for dictionaries to avoid prototype, or freeze Object.prototype. This is advanced and specific to JS.
- **How do you securely store passwords in a Node application?** – You don't store raw passwords, you store a salted hash (using bcrypt or Argon2, etc.). This might seem more general security than JS-specific, but a full-stack JS dev should know: use a strong hashing algorithm with a salt and proper work factor. Also mention not to ever log or send the plaintext password. While not in the category list explicitly, it's a critical security knowledge point.
- **What is Cross-Site Script Inclusion (XSSI)?** – An advanced one: If a site returns JSON and doesn't protect it, another site could include that JSON via a `<script src="victim.com/data.json">` tag. The JSON would be parsed as JS, potentially exposing it (if the attacker can wrap it in known JS). One mitigation is to prepend `)]}'` (Angular does this) to JSON responses to make them invalid as JS. This is more niche but interesting if someone mentions it.
- **How would you secure a REST API built with Node/Express?** – Look for things like: using HTTPS, requiring authentication (tokens, API keys, etc.), rate limiting to prevent brute force or abuse (maybe using middleware), validating all inputs (to prevent injections or crashes), enabling CORS only for allowed domains if it's a public API, perhaps also using Helmet to set security headers (even for API, maybe content-type options, etc.). Essentially a summary of backend security practices beyond just XSS/CSRF (since those are more front-end user-facing issues).

15. TypeScript and Static Typing

Basic

- **What is TypeScript and how is it different from JavaScript?** – TypeScript is a superset of JavaScript that adds optional static typing and other features (interfaces, generics, enums, etc.) and then compiles down to plain JS [74] . Emphasize that any valid JS is valid TS (just with type annotations possibly) and TS needs a compilation step. It helps catch errors at development time (e.g., mis-typing a property name) that would otherwise only show up at runtime in JS [75] .
- **What are the advantages of using TypeScript?** – Common points: static type checking (catches many bugs early) [75] , better IDE support/intellisense and documentation since types are known, easier refactoring with confidence, and the ability to use modern JS features (since TS compiler can target older runtimes). Possibly mention that large codebases are easier to maintain when you have types, as you can understand data structures and APIs quickly.
- **How do you declare a variable with a specific type in TypeScript?** – For example: `let age: number = 30;` or `const name: string = "Alice";`. Or mention type inference (if you do

`let age = 30`, TS infers it's a number, so often you don't need explicit annotation). Possibly mention other basic types like boolean, array (`number[]`), etc. This ensures they actually wrote some TS.

- **Can TypeScript run directly in the browser?** – No, browsers don't understand TS; it must be compiled/transpiled to JavaScript first. This is an important distinction: the build step. Also mention that similarly Node needs either transpilation or recent Node supports some TS via loaders but generally also needs compile.
- **What is a `.d.ts` file?** – It's a TypeScript declaration file (ambient type declarations) which provides type information for JavaScript libraries or for separated compile outputs. They often come with libraries (DefinitelyTyped) to describe the shape of modules. This is a bit advanced for basic, but sometimes even beginners see them when using JS libraries in TS. So maybe intermediate. Alternatively, ask: *"How do you get types for a library like jQuery in TS?"* expecting "via @types packages which provide .d.ts."

Intermediate

- **What is an interface in TypeScript?** – Interfaces allow you to define the shape of an object (properties and their types) or function signatures. For example: `interface Person { name: string; age: number }` and then you can use that to type objects. Also mention that classes can implement interfaces. They are a way to enforce certain structure. (Could also mention type aliases as similar, maybe difference).
- **What are generics in TypeScript?** – Generics are a way to create reusable components that work with multiple types while still preserving type information. For instance, `function identity<T>(value: T): T { return value; }` can take any type T and return that type T. Or a generic class `Array<T>` where T is the element type. They allow writing code that can adapt to types (like a LinkedList<T> that can be a list of any type). Without generics, you might use `any`, which loses type info.
- **How does TypeScript help with IDE autocompletion?** – Because TS knows the types of variables and the shape of objects, the IDE can offer suggestions for properties/methods (`obj.` will show you all properties TS knows obj has). It also can show you documentation if provided in types, and catch mistakes like calling a method that doesn't exist. Essentially, static types enable smarter tooling.
- **What is the difference between `any` and `unknown` types in TS?** – `any` basically opts-out of type checking – you can do anything with an `any` and TS won't complain (but you also lose type safety). `unknown` is a safer counterpart: you can assign anything to `unknown`, but you can't use an `unknown` value without narrowing its type (e.g., via typeof check or type guard) [74]. It forces you to do a runtime check before using it. So `unknown` is better for APIs that could return anything – it requires checking, whereas any could lead to runtime errors if misused.
- **Can you give an example of a union type and an intersection type in TypeScript?** – Union: a variable that can be one of several types, e.g. `let result: string | number;` means result can be either string or number. Intersection: combine types, e.g. `type PersonAccount = Person & Account;` which means an object that has all properties of Person and Account. Possibly mention a use-case of union (function returns string or Error) and intersection (mixing multiple interfaces). This shows knowledge of type composition.

Advanced

- **What are TypeScript decorators, and what can they be used for?** – Decorators are an experimental feature (stage 2 in TC39) that TS (with `"experimentalDecorators": true`) allows. They're functions that can annotate classes, methods, properties, etc., providing meta-programming capabilities (like adding metadata or altering behavior). Used in Angular

extensively (e.g. @Component, @Injectable are class decorators). In TS, since not standard, it's mostly for frameworks. If they know: a decorator is basically a function that gets the target and possibly property name etc. and can define things like property descriptors.

- **Explain conditional types in TS (e.g., `T extends U ? X : Y`).** – Conditional types allow type logic: for example `type NonNull<T> = T extends null | undefined ? never : T;`. This is advanced, but shows up in lib definitions (like ReturnType, Exclude, etc., are implemented with conditional types). If the candidate can explain that, it's impressive: basically a form of pattern matching on types to decide another type. They might mention `infer` keyword usage inside conditional types as well, which is even more advanced.
- **How does TypeScript's type inference work?** – They might describe that TS can infer types from assignments and return statements. For instance, `let x = 5;` infers `x` as number. In functions, if you return a string, TS infers the function's return type as string if not explicitly annotated. And inference flows: if you pass a literal to a generic, it infers the type parameter. Understanding inference matters for designing good APIs that don't require a lot of explicit typing.
- **What is the `as const` assertion in TS?** – When you write `let arr = [1,2,3] as const;`, it will make the type of arr be readonly [1,2,3] (tuple of literal 1,2,3) rather than number[]. Essentially, `as const` tells TS to infer the most specific literal types and mark as readonly. It's used for ensuring that object literals have literal type properties (especially helpful for making sure a literal matches a more specific type or for something like union of literal strings).
- **How do you approach migrating a large JavaScript codebase to TypeScript?** – This is more process oriented. Good answers: do it incrementally, possibly using `allowJs` to mix TS and JS, start by adding JSDoc type annotations in JS as a stepping stone or use `checkJs`. Or rename files gradually to .ts and fix errors, perhaps starting with core modules. Also, possibly set TS config `noImplicitAny: false` at first to ease in, then tighten later. This shows practical experience with TS beyond just new projects.

16. System Design with JavaScript/TypeScript (full-stack architecture, API design)

Basic

- **What is a RESTful API and how do you design one?** – RESTful API uses HTTP methods (GET, POST, PUT, DELETE, etc.) and resource-based URIs to perform operations on resources (e.g. `/users` endpoint). Designing one means defining clear endpoints for resources, using proper methods for actions (GET for read, POST for create, etc.), and maybe proper status codes in responses [76]. The question expects that they know general principles like statelessness (each request has all info needed, server doesn't keep client state between requests) and maybe using JSON for data format.
- **Explain client-server architecture in a web application.** – The client (frontend, usually running in browser) makes requests to the server (backend) for data or to perform actions. The server processes those requests (maybe talking to a database) and returns responses. They may mention that in a full-stack JS context: client could be an SPA built with React/Angular, server could be Node/Express serving a REST API, and they communicate over HTTP (often JSON).
- **What is the difference between *frontend* and *backend* development?** – Frontend is the UI in the browser (HTML/CSS/JS that the user interacts with). Backend is the server-side logic, database interactions, authentication, business rules, etc., typically not directly seen by the user but powering the app. This is basic, to ensure understanding of roles in system architecture.

- **What does it mean for a server to be stateless?** – In context of REST, stateless means each request is independent – the server does not store information about the state of the client between requests. If needed, state is sent (like a token or parameters) with each request. This simplifies scaling (because any server can handle any request, no session stickiness needed) [76] . They might relate statelessness to how REST works (and how cookies/sessions are an example of stateful if not careful).
- **Why might you use JSON for data exchange in a web system?** – JSON is lightweight, human-readable, and natively understood by JavaScript (since it's basically JS object syntax). It's become the de-facto standard for APIs because it's easy to produce and consume in web and mobile. Alternative formats (XML, etc.) are heavier or need more parsing code. So JSON hits sweet spot for structured data exchange.

Intermediate

- **How would you design the architecture for a web application using React on the frontend and Node.js on the backend?** – They should talk through separation of concerns: The React app (possibly built and served statically or via CDN) communicates with the Node backend via HTTP (REST API or GraphQL). The Node backend handles business logic, database queries (maybe an ORM, etc.). Possibly mention using a reverse proxy or load balancer if scaling horizontally. Also, maybe mention security (auth tokens stored in frontend, validated in backend). It's checking if they see the big picture – front/back comms, not heavy details.
- **What is GraphQL and how does it differ from REST?** – GraphQL is a query language for APIs where the client can request exactly the data it needs, and get it in a single response (no over/under-fetching). It uses a single endpoint and a schema that defines types and queries/mutations. REST uses multiple endpoints and fixed data shapes. GraphQL can reduce number of requests (client can combine data needs in one call) and gives more flexibility to frontends, but adds complexity of maintaining a schema and resolver logic on backend. They should understand pros/cons vs REST.
- **How do you handle user authentication in a modern web app (JWT vs sessions)?** – They should mention two common approaches: Sessions (server keeps track of logged-in users via session store, and a cookie (often HttpOnly) is used to identify the session – classical approach) vs JWT tokens (stateless, token is issued to client after login, client sends it with each request (usually in Authorization header or cookie), server verifies token signature and trusts the claims inside). Compare: JWT is stateless, easier to scale horizontally (no session store needed), but revocation is hard; sessions can be easily invalidated from server but need a store and sticky sessions if multiple servers (unless using a centralized store like Redis). This assesses knowledge of system auth design.
- **How would you design a system to handle real-time updates to clients (e.g. a chat app)?** – Look for mention of WebSockets (or alternatives like Server-Sent Events or long polling). Likely they'll say: Use WebSocket connections between client and server for push messaging in real-time. Node can maintain WebSocket connections (maybe using libraries like Socket.io which simplifies fallback and pub/sub). Or if they mention newer stuff like WebRTC for P2P (for chat maybe not needed unless video/voice). This tests knowledge of beyond request/response, and event-driven architecture.
- **What are microservices and would you use Node.js to build one?** – Microservices are an architectural style where the application is split into small, independent services each responsible for a specific domain or functionality, communicating over the network (usually HTTP/REST or messaging). Node.js can be used to build microservices – because Node is good for lightweight, network-heavy tasks, you could have many small Node services. They might discuss pros: better scalability, can deploy/update independently, polyglot possible, each service simpler. Cons: added complexity in devops, communication overhead, distributed system

challenges (network failure, data consistency). Check if they grasp when microservices might be appropriate vs a monolith.

Advanced

- **How would you scale a Node.js backend to handle a high load of requests?** – They should talk about both vertical and horizontal scaling. Horizontal: run multiple Node instances (cluster mode on one machine to utilize multi-core, and multiple machines behind a load balancer). Possibly containerization (Docker/Kubernetes) to manage scaling. Maybe mention using a reverse proxy like Nginx or cloud LB to distribute to Node processes. Also mention optimizing the app code (caching results, using a CDN for static assets, etc.), and using asynchronous approaches for I/O to keep event loop free (which Node does by default mostly). Also possibly splitting services (microservices) if one part is the bottleneck. This shows understanding of scaling concepts in Node context.
- **What considerations go into designing an API for a large scale application?** – Could include versioning (how do you version your API so changes don't break clients), authentication/authorization (OAuth, JWT, etc.), rate limiting and throttling (to protect from abuse), pagination of data for large lists, idempotency of certain operations, using appropriate HTTP status codes and methods, documentation (maybe using OpenAPI/Swagger). Also error handling and consistency in responses (structured error messages). Essentially designing an API is not just endpoints, but also making it robust and future-proof.
- **How would you integrate a third-party service or API into your application design?** – Look for mention of making the calls from the backend vs frontend depending on needs (e.g., hide API keys by calling from backend). Also handling failures (retry logic, circuit breaker pattern to disable calls if the service is down to avoid waiting on timeouts, fallback strategies). Possibly mention using webhooks vs polling if third-party provides them (for receiving updates). Security: storing credentials safely (not in client), etc. This shows system design considering external dependencies.
- **Can you outline how a full-stack TypeScript application might share code between frontend and backend?** – If they know advanced stuff: They might mention using a monorepo and a tool like Nx or lerna, having a folder with common types (interfaces, maybe validation schemas) that both frontend and backend import. This way, the data contracts (like a User type) are single-sourced. They might mention GraphQL with TypeScript, generating types from schema for front and back. This tests if they've thought about consistency and code reuse in TS across the stack.
- **What is serverless architecture and how can JavaScript/TypeScript be utilized in it?** – Serverless (like AWS Lambda, Azure Functions) means you write functions that run on demand, you don't manage servers. JS/TS is commonly supported in these (Node.js runtime). They could describe a scenario: instead of a always-on server, you have a set of Lambdas – one for each API endpoint, behind API Gateway. Benefits: scaling is handled by cloud, pay-per-use, etc. Challenges: cold starts, statelessness (each invocation isolated). It's modern architecture knowledge.

---

1 6 7 9 17 18 21 40 41 46 90+ Essential Javascript Interview Questions and Answers
https://www.simplilearn.com/tutorials/javascript-tutorial/javascript-interview-questions

2 3 4 8 10 11 12 13 14 15 16 19 20 22 26 28 29 30 31 32 34 35 36 37 43 45 49
JavaScript Interview Questions (Quiz) for Front End Interviews | The Official Front End Interview Handbook 2025
https://www.frontendinterviewhandbook.com/javascript-questions

5 JavaScript Interview Questions and Answers (2025) - InterviewBit
https://www.interviewbit.com/javascript-interview-questions/

23 24 25 27 33 42 44 53 54 55 56 57 58 59 Top 100+ Node.js Interview Questions and Answers for 2025
https://www.simplilearn.com/tutorials/nodejs-tutorial/nodejs-interview-questions

38 JavaScript Optional Chaining - GeeksforGeeks
https://www.geeksforgeeks.org/javascript/javascript-optional-chaining/

39 Only a Pro can answer these JavaScript interview questions
https://javascript.plainenglish.io/only-a-pro-can-answer-these-javascript-interview-questions-eb665ef7580e

47 48 60 63 64 65 66 67 69 71 76 Some Good Interview Questions for Experienced Frontend Developers | TypeOfNaN
https://typeofnan.dev/some-good-interview-questions-for-experienced-frontend-developers/

50 Top 12 interview question for a React Developer - DEV Community
https://dev.to/brdnicolas/top-12-interview-question-for-a-react-developer-34f1

51 How does virtual DOM in React work? What are its benefits and ...
https://www.greatfrontend.com/questions/quiz/how-does-virtual-dom-in-react-work-what-are-its-benefits-and-downsides

52 48 React interview questions with answers | by Dzmitry Ihnatovich
https://medium.com/@ignatovich.dm/48-react-interview-questions-with-answers-084cb8ce706d

61 62 Top 50 Jest Interview Questions and Answers
https://www.lambdatest.com/learning-hub/jest-interview-questions

68 72 Explain the same-origin policy with regards to JavaScript
https://www.greatfrontend.com/questions/quiz/explain-the-same-origin-policy-with-regards-to-javascript

70 73 Devinterview-io/web-security-interview-questions - GitHub
https://github.com/Devinterview-io/web-security-interview-questions

74 Most Common TypeScript Interview Questions for Frontend ...
https://dev.to/suraj_kumar_fb57ae0928df2/most-common-typescript-interview-questions-for-frontend-developers-5cpo

75 50 TypeScript Interview Questions and Answers for 2025
https://www.simplilearn.com/tutorials/typescript-tutorial/typescript-interview-questions